

Specifica algebrica del tipo estratto Albero

Specifica sintattica:

Operatori:

crea() → Albero // crea un albero binario vuoto

aggiungiR(Albero, Nodo) → Albero // aggiunge la radice all'albero

aggiungiS(Albero, Nodo, Nodo) → Albero // aggiunge il secondo nodo come figlio sinistro del primo nodo nell'albero

aggiungiD(Albero, Nodo, Nodo) → Albero // aggiunge il secondo nodo come figlio destro del primo nodo nell'albero

conta(Albero) → Intero // conta il numero di nodi nell'albero

max(Albero) → Intero // restituisce la profondità massima dell'albero

pota(Albero, Nodo) → Albero // cancella il sottoalbero radicato nel nodo trasformandolo in foglia; solleva errore se il nodo è assente

Equal(Albero, Albero) → Boolean // restituisce vero se i due alberi hanno uguale profondità massima, falso altrimenti

I costruttori sono:

- crea()
- aggiungiR(Albero, Nodo)
- aggiungiS(Albero, Nodo, Nodo)
- aggiungiD(Albero, Nodo, Nodo)

Specifica semantica:

declara: $e, e': \text{Albero}; n, r, s, d, n', r', s', d': \text{Nodo}$

$$\text{conta}(\text{crea}()) = 0$$

$$\text{conta}(\text{aggiungiR}(e, r)) = 1$$

$$\text{conta}(\text{aggiungiS}(e, n, s)) = 1 + \text{conta}(e)$$

$$\text{conta}(\text{aggiungiD}(e, n, d)) = 1 + \text{conta}(e)$$

Assumo $+ : (\text{Intero}; \text{Intero}) \rightarrow \text{Intero}$

$\sim \quad \sim \quad \sim \quad \sim \quad \sim \quad \sim$

Definiamo:

$\text{prof}(\text{Albero}, \text{Nodo}) \rightarrow \text{Intero}$

$$\text{prof}(\text{crea}()) = \text{error}$$

$$\text{prof}(\text{aggiungiR}(e, r), n') = \text{if } n' = r \text{ then } 0 \text{ else error}$$

prof(aggiungi: S(e, n, s), n') = if s = n' then 1 + prof_n(e, n)
 else prof_n(e, n')

prof(aggiungi: D(e, n, d), n') = if d = n' then 1 + prof_n(e, n)
 else prof_n(e, n')

mex(crea()) = error

mex(aggiungi: R(e, r)) = 0

mex(aggiungi: S(e, n, s)) =

= if 1 + prof(e, n) > mex(e) then
 1 + prof(e, n)
 else
 mex(e)

mex(aggiungi: D(e, n, d)) =

= if 1 + prof(e, n) > mex(e) then
 1 + prof(e, n)
 else
 mex(e)

~ ~ ~ ~ ~

pote(crea(), n') = error

pote(aggiungi: R(e, r), n') = if n' = r then e
 else error

pote(aggiungi: S(e, n, s), n') = if n' = s then aggiungi(e, n, s)
 else
 if n' = n then
 else pote(e, n')

pote(aggiungi: D(e, n, d), n') = if n' = d then aggiungi(e, n, d)
 else
 if n' = n then
 else pote(e, n')

~ ~ ~ ~ ~

$$\text{Equal}(\text{cree}, \text{cree}) = \text{true}$$

$$\text{Equal}(\text{cree}, \text{aggiungiB}(e', r')) = \text{false}$$

$$\text{Equal}(\text{cree}, \text{aggiungiS}(e', n', s')) = \text{false}$$

$$\text{Equal}(\text{cree}, \text{aggiungiD}(e', n', d')) = \text{false}$$

$$\text{Equal}(\text{aggiungiB}(e, r), \text{cree}) = \text{false}$$

$$\text{Equal}(\text{aggiungiB}(e, r), \text{aggiungiB}(e', r')) = \text{true}$$

$$\text{Equal}(\text{aggiungiB}(e, r), \text{aggiungiS}(e', n', s')) = \text{false}$$

$$\text{Equal}(\text{aggiungiB}(e, r), \text{aggiungiD}(e', n', d')) = \text{false}$$

$$\text{Equal}(\text{aggiungiS}(e, n, s), \text{cree}) = \text{false}$$

$$\text{Equal}(\text{aggiungiS}(e, n, s), \text{aggiungiB}(e', r')) = \text{false}$$

$$\text{qual}(\text{aggiungiS}(e, n, s), \text{aggiungiS}(e', n', s')) =$$

$$= \text{max}(\text{aggiungiS}(e, n, s)) = \text{max}(\text{aggiungiS}(e', n', s'))$$

$$\text{Equal}(\text{aggiungiS}(e, n, s), \text{aggiungiD}(e', n', d')) =$$

$$= \text{max}(\text{aggiungiS}(e, n, s)) = \text{max}(\text{aggiungiD}(e', n', d'))$$

$$\text{Equal}(\text{aggiungiD}(e, n, s), \text{cree}) = \text{false}$$

$$\text{Equal}(\text{aggiungiD}(e, n, s), \text{aggiungiB}(e', r')) = \text{false}$$

$$\text{qual}(\text{aggiungiD}(e, n, s), \text{aggiungiS}(e', n', s')) =$$

$$= \text{max}(\text{aggiungiD}(e, n, s)) = \text{max}(\text{aggiungiS}(e', n', s'))$$

$$\text{Equal}(\text{aggiungiD}(e, n, s), \text{aggiungiD}(e', n', d')) =$$

$$= \text{max}(\text{aggiungiD}(e, n, s)) = \text{max}(\text{aggiungiD}(e', n', d'))$$

